Eötvös Loránd Tudományegyetem

Informatikai Kar

Algoritmusok és Alkalmazásaik Tanszék

Cache-optimális algoritmusok elemzése

Szabó László Ferenc
Docens

Nagy Péter
Programtervező Informatikus MSc

Budapest, 2023

Tartalomjegyzék

1. Bevezetés **3**

2. Cache modellje **4**

2.1. Csere stratégiák 5

2.1.1. Online algoritmusok 6

2.1.2. Verem stratégiák 6

2.1.3. FIFO 7

2.1.4. LRU 7

2.2. Asszociativitás 8

2.3. Írási stratégia 9

3. Gyorsítótár független algoritmusok **11**

3.1. Lista 11

3.1.1. Másolás 11

3.1.2. Partíció 11

3.1.3. Megszámlálásos rendezés 11

3.2. Rendezés 11

3.3. Mátrix szorzás 11

3.4. FFT 12

3.5. Diff.egyenlet közelítés 12

3.6. Statikus keresőfa 12

3.7. Edénysor szita 12

3.7.1. A rekurzív fa 12

3.7.2. Feladat 13

3.7.3. A megoldás 13

3.7.4. Műveletigény gyorsítótár nélkül 15

3.7.5. Cache-oblivious edényszita 18

3.7.6. Műveletigénye gyorsítótárral 18

3.7.7. Feltételek a gyorsítótár méretére 18

3.7.8. Tetszőleges n-ig, nem csak teljes fára 18

3.7.9. Az igazi program fix méretű számokkal 18
3.7.10. Az igazi program változó méretű számokkal 18

4. Szimulátor **19**
4.1. Igazi cache 19
4.2. Megoldás 19
4.3. Választható cache-ek 19

5. Mérési eredmények **20**

6. Irodalomjegyzék **21**

1. fejezet

Bevezetés

A modern számítógépek olyan sebességeket érnek el, hogy a jelterjedés fizikai korlátai már egy integrált áramkörön belül sem feltétlenül engedik meg az összes áramköri elem szinkron működést. A végrehajtó egység és a memória sebessége között már nagyságrendi különbségek vannak.

A probléma kezelésére gyorsítótárakat vezettek be a végrehajtó egység és a memória közé, ami az utoljára használt memóriaelemeket tárolja. A gyorsítótár sebessége nagyobb mint a memóriáé, de lassabb, mint a végrehajtó egységé. Egy modern számítógépben a gyorsítótáraknak hierarchiája is van, ahol egy gyorsítótár egy szintben alatta lévő másik gyorsítótár elemeit tárolja.

A gyorsítótárakkal ellátott számítógép legjobb kihasználásához olyan algoritmusokat kell tervezni, amik figyelembe veszik a gyorsítótárak hierarchiáját.

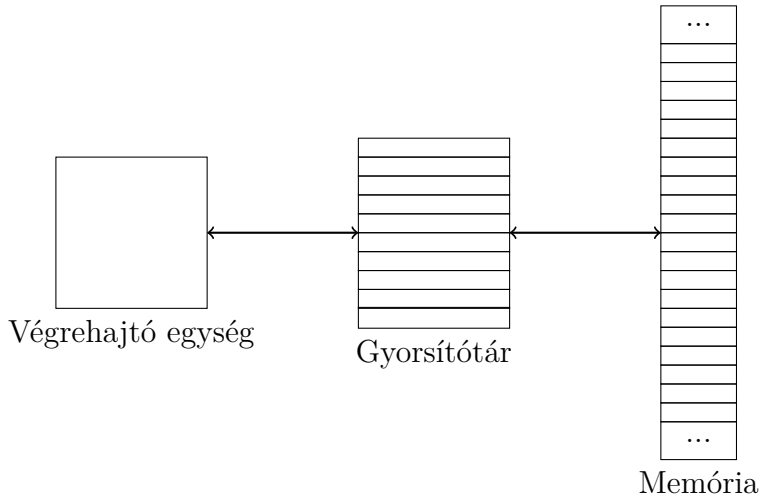
2. fejezet

Cache modellje

Katasztrofális.

Az ideális gyorsítótár modell[1] egy végrehajtó egységből, memóriából, és egy gyorsítótárból áll. A végrehajtó egység a memóriát csak a gyorsítótáron keresztül éri el.

A végrehajtó egység a memóriát gépi szavanként éri el, ez egy kicsi konstans, például 8 bájt. Egy (Z, L) gyorsítótár L hosszú gyorsítótársorokból áll, és Z/L sort tartalmaz. Egy sor L gépi szót tárol, ezek a memóriában folytonosan helyezkednek el. Egy sor címe a memóriába csak a sorhossz többszöröse lehet. A gyorsítótár a memóriát gyorsítótársoronként tárolja. Egy sor tipikusan több gépi szóból áll, és ezek a szavak folytonosan helyezkednek el a memóriában. Egy sor címe a memóriában csak a sorhossz többszöröse lehet. Egy sor a gyorsítótárban a memória bármelyik szakaszát tárolhatja.



2.1. ábra. Az ideális gyorsítótár modell

Amikor a végrehajtó egység el akar érni egy adott szót a memóriában, a gyorsítótárhoz fordul. Ha a tárban van olyan sor, ami tartalmazza a kért gépi szót, akkor

az kérést azonnal kiszolgálja. Ez a cache hit. Ha a tár egyik sora se tartalmazza a kért szót, akkor a szóhoz tartozó sort a tár be kell olvassa a tárba, majd ezután tudja teljesíteni a kérést. Ez a cache miss.

Ha a tárban nincs üres sor amikor egy új sort be kell olvasni, akkor először egy sort ki kell ürítsen. Ha az ürítésre kiválasztott sor módosult a hozzá tartozó memóriához képest, akkor azt vissza kell írnia a memóriába. Miután a sort szinkronizálta a memóriával, az új sorral felülírja az ürített sort.

A gyorsítótár-csere stratégiája diktálja, hogy a tár hogyan választja ki a sort, amit ürít. Az ideális gyorsítótár modell ürítési stratégiája az optimális offline stratégia[2]. Az optimális stratégia azt a sort üríti, amit a jövőben legkésőbb ér el a program a teljes futása során. Ez a stratégia használja ki a legjobban az időbeli lokalitást.

A gyorsítótár modellben az időbonyolultság az algoritmus futása alatt végzett elemi műveletek száma, a memóriaelérésekkel együtt. A gyorsítótár-bonyolultság a futás alatti találati hibák száma.

Egy algoritmus gyorsítót-tudatos, ha van olyan fordítási vagy futási idejű paramétere, ami a gyorsítótár méretétől függ. Ha nincs ilyen paramétere, akkor az algoritmus gyorsítótár-független.

2.1. Csere stratégiák

Katasztrófális.

Több csere startégia is ismert:

- MIN, minimális, OPT, optimális, vagy Béládv algoritmusa: Ezt az elemet cseréli, amelyiket a jövőben legkésőbb éri el az algoritmus. Ennek eldöntésére szükség van a teljes memóriaelérési sorozatra.

- LRU, least recently used, legrégebben használt: Azt a sort cseréli, amit a legrégebben ért el az algoritmus. Ennek eldöntésére elég a csere pillanatáig ismerni az elérési sorozatot. Az LRU heurisztikája az a feltételezés, hogy az időben közelebbi elért memóriát valószínűleg újra el kell érni.

- LFU, least frequently used, legritkábban használt: A gyorsítótár nyilván tartja, hogy egy sort hányszor ért el az algoritmus mióta a gyorsítótárba került. Azt az elemet cseréli, amelyik elérési száma a legkisebb.

- FIFO, first in, first out, első be, első ki: Azt a sort cseréli, amelyik legrégebben került a tárba. A stratégia előnye az egyszerű megvalósíthatóság.

- Random, véletlen: A cserét véletlenszerűen választja ki. Az előnye az egyszerű implementáció.

Ha S egy gyorsítótár-csere stratégia, akkor legyen S(k) egy olyan gyorsítótár, ami az S stratégiát használja, és k sort tartalmaz.

2.1.1. Online algoritmusok

Katasztrofális.

A gyakorlatban megvalósítható gyorsítótár-csere stratégiák online algoritmusok. Az online algoritmusok a bemenetüket sorban, egyesével kell feldolgozzák, a teljes bementet nem látják egyszerre. Egy gyorsítótár-csere algoritmusnak minden memóriaeléréskor úgy kell döntést hoznia a cseréről, hogy csak a múltbeli elérések sorozatát ismeri.

Egy offline algoritmus a teljes bemenethez hozzáfér a teljes futása során.

A stratégiák elemzését egy egyszerűbb modellben[5] végezzük. Legyen $A = [1, 2, ..., n]$ az elérhető elemek halmaza. Legyen X egy A feletti véges sorozat, az elérések sorozata. Legyen $B_t(k)$ az $S(k)$ gyorsítótár tartalma X t hosszú prefixének feldolgozása után. Minden k-ra:

- Legyen $B_0(k) = \emptyset$.
- Ha $t > 0$, és $X_t \in B_{t-1}(k)$, akkor legyen $B_t k = B_{t-1}(k)$. Ez egy találat.
- Ha $t > 0$, $X_t \notin B_{t-1}(k)$, és $|B_{t-1}(k)| < k$, akkor legyen $B_t k = B_{t-1}(k) \cup \{X_t\}$. Ez egy találati hiba, ürítés nélkül. Ha $t > 0$, $X_t \notin B_{t-1}(k)$, és $|B_{t-1}(k)| = k$, akkor legyen $y_t \in B_{t-1}(k)$, és $B_t k = B_{t-1}(k) \setminus y_t \cup \{X_t\}$. Ez egy találati hiba, ürítéssel. y_t az $S(k)$ stratégiája választja ki, ez az ürített elem.

Ebben a modellben a gyorsítótárban nem sorok vannak, hanem A elemei. Ezek a gyorsítótár modellben a memória azon tartományai, amik gyorsítótár sorok lehetnek. Valamint az elérésben nem különböztetjük meg az olvasást és az írást.

2.1.2. Verem stratégiák

Katasztrofális.

Egy S gyorsítótár-csere stratégiát verem algoritmusnak[5] nevezünk, ha teljesíti a következő feltételt. S verem algoritmus, ha minden $t \in [0, 1, ..., |X|]$, és minden $k \in \mathbb{N}$-re

$$B_t(k) \subset B_t(k + 1)$$

Azaz ha minden időpontban egy nagyobb gyorsítótár tartalmazza az összes kisebb gyorsítótár elemeit.

A verem algoritmusnak ekvivalens definíciója az, ha S X futtatása közben minden t időpontra ad egy egyértelmű preferencialistát az X-ben előforduló elemekre, a gyorsítótár méretétől függetlenül, és egy $S(k)$ gyorsítótár csere esetén mindig az éppen tartalmazott elemei közül a legkisebb precedenciájút üríti.

Egy verem algoritmust használva a gyorsítótár méretét növelve a találtai hibák száma nem nő. Nézzünk t időpontban egy gyorsítótár találatot $S(k)$-ban. Mivel $B_t(k) \subset B_t(k+1)$, ezért az elérés $S(k+1)$-ben is találat. Ez indukcióval implikálja, hogy minden k-nál nagyobb gyorsítótárban is találat az elérés.

Az LRU verem algoritmus.

Az OPT verem algoritmus. Lehetne példákat adni, hogy a demand paging elhagyásával jobb eredményt ér el az OPT. A store is másmilyen, ha nem kell a cache-be tenni, hanem no-allocate is lenne. Példa/ábra.

Az LFU verem algoritmus, feltéve, hogy az egyenlő frekvenciájú elemekre létezik egy egyértelmű precedencia.

A FIFO nem verem algoritmus.

A véletlen startégi nem feltétlen verem algoritmus. Ha egyenletesen véletlenül választjuk ki egy $S(k)$ gyorsítótár elemét ürítésre, akkor pozitív a valószínűsége annak, hogy az $S(k+1)$ tár egy másik elemet választ. Ha a tár méretétől egyenletes véletlenül X egyedi elemeinek egy permutációját, és ezt a precedencialistát használjuk az ürített elem kiválasztására, akkor a véletlen stratégia verem algoritmus. Ez az RR startégia. Egy adott $RR(k)$ gyorsítótárban egy elem ürítésének esélye $1/k$, tehát egyenletes eloszlású.

2.1.3. FIFO

Katasztrofális.

Béládyi[3] megmutatta, hogy FIFO stratégiára létezik olyan k és K, $k < K$, és egy elérési sorozat, hogy ezen az elérési sorozaton a FIFO(K) több találati hibát eredményez, mint a FIFO(k)-n. Fornai és Iványi megmutatta[4], hogy a hiba arányának nincs felső korlátja.

2.1.4. LRU

Katasztrofális. k-competetive hivatkozás.

Egy adott problémára a versenyképességi analízisben egy A online algoritmust hasonlítunk össze az O optimális offline algoritmussal. Legyen X egy inputsorozat, $A(X)$ az A műveletigénye X-en, és $O(X)$ az O műveletigénye X-en. Ekkor A k-versenyképes, ha létezik egy C konstans, hogy minden X inputsorozatra

$$A(X) \leq kO(X) + C$$

Slater és Tarjan[6] megmutatta, hogy az LRU(k) az OPT(k^*)-hoz képest $k/(k-k^*+1)$ versenyképes. Ha $k = k^*$, akkor az LRU k-versenyképes az OPT-hoz képest. Ha $k = 2k^*$, akkor az LRU(k) az OPT(k*)-hoz képest

$$k/(k-k^*+1) = 2k^*/(2k^*-k^*+1) = 2k^*/(k^*+1) < 2$$

versenyképes[1].

Legyen A egy adott algoritmus. Legyen $LRU(n, Z, L)$ a gyorsítótár komplexitása A-nak egy n méretű bemeneten és egy LRU(Z, L) gyorsítótáron. Legyen $OPT(n, Z, L)$ a gyorsítótár komplexitása A-nak egy n méretű bemeneten és egy OPT(Z, L) gyorsítótáron. Ekkor az $LRU(n, Z, L)$ gyorsítótár komplexitás reguláris[1], ha

$$LRU(n, Z, L) = O(LRU(n, 2Z, L))$$

Tegyük fel, hogy A egy reguláris gyorsítótár komplexitású algoritmus. Ekkor

$$OPT(n, Z, L) \leq LRU(n, Z, L) = O(LRU(n, 2Z, L)) = O(OPT(n, Z, L)$$

az optimalitást, a regularitást, és a 2-versenyképességet felhasználva. Tehát

$$LRU(n, L, Z) = \Theta(OPT(n, L, Z)$$

2.2. Asszociativitás

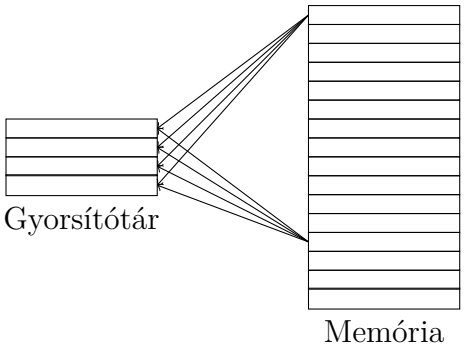
Katasztrofális.

Az ideális gyorsítótár modellben a memória egy sorát a gyorsítótár bármelyik sorába be lehet hozni. Ezt hívják teljesen asszociatív gyorsítótárnak.

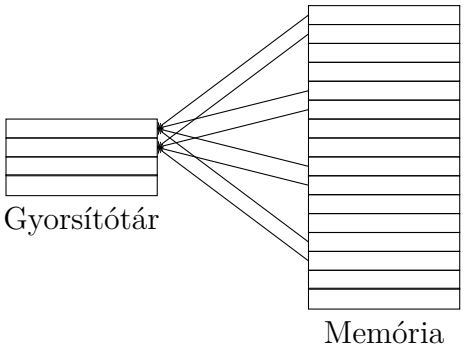
A gyakorlatban egy teljesen asszociatív gyorsítótár csak kevés sorra valósítható meg hatékonyan, mert minden elérésnél a gyorsítótárnak meg kell keresnie a sorok között, hogy tárolja-e az elérni kívánt sort.

Közvetlen leképezésű gyorsítótárban a memória egy sora a tárban csak egyetlen sorba hozható be. Ez csere stratégiát is meghatározza, a beolvasni kívánt sor címe meghatározza az ürített sort. Általában egy sor címét a gyorsítótárban a sor címének alsó bitjei határozzák meg, így a gyorsítótárba lehet hozni a memória egy egybefüggő szakaszát.

Csoport-asszociatív gyorsítótárban a memória egy sora a gyorsítótár több sorába is behozható, de nem az összesbe. A gyorsítótár több, egymástól független teljesen asszociatív gyorsítótárból áll. A memória sor címe meghatározza, hogy a sor melyik asszociatív tárba kerül, és az asszociatív táron belül a csere stratégia alapján kerül

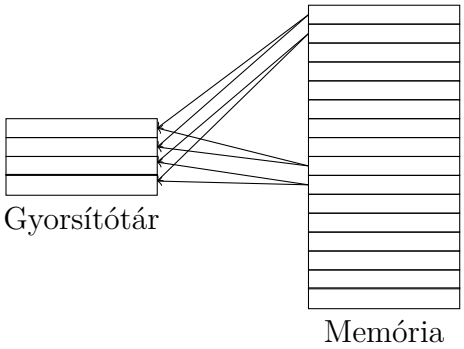


2.2. ábra. Teljesen asszociatív gyorsítótár



2.3. ábra. Közvetlen leképezésű gyorsítótár

elhelyezésre.



2.4. ábra. Csoport-asszociatív gyorsítótár

Ide lehetne tenni részleteket a cím bitjeiről, meg helykitöltő ábrákat. Talán van haszna a szimulátor tárgyalásánál.

2.3. Írási stratégia

Katasztrofális. Bele lehet ebbe keverni a memória tényleges címezhetőségét. Nem tudom, hogy ide akarom-e másolni a wikipedia-ról a hivatkozást a tipikus write back-write allocate és write through-no write allocate-re.

A gyorsítótár az írásokat többféleképpen is kezelheti.

A write back stratégia az írást csak a gyorsítótárban végzi el. Ebben az esetben a gyorsítótárnak a sorok módosítottságát is nyilván kell tartani, és egy módosított sort ürítés előtt a memóriába kell írnia.

A write through stratégia az írást a gyorsítótárban is elvégzi, és a memóriában is.

Írási találtai hiba esetén további két stratágiát lehet választani. A write allocate startégia találati hiba esetén a módosítandó sort a gyorsítótárba tölti, szükség esetén egy sort ürítve. A no-write allocate az írást közvetlenül a memóriában végzi el.

3. fejezet

Gyorsítótár független algoritmusok

3.1. Lista

Katasztrofális.

Ha a memóriában nem túl sok egybefüggő részen vannak sorban a lista elemei, akkor az cache-oblivious végigolvasni. A cache line méretével oszthatjuk a sebességet.

3.1.1. Másolás

3.1.2. Partíció

3.1.3. Megszámlálásos rendezés

3.2. Rendezés

Katasztrofális.

Oszd meg és uralkodj. 2 algoritmus is van. Az alsó korlát elérhető, de a papír nem az igazi erről. Tall cache assumption. A tall cache assumption lényegesen befolyásolja a műveletigényt.

3.3. Mátrix szorzás

Katasztrofális.

Oszd meg és uralkodj.

3.4. FFT

Katasztrofális.

Oszd meg és uralkodj. Alsó korlát is van, amit elér, de túl erős feltevésekkel.

3.5. Diff.egyenlet közelítés

Katasztrofális.

Oszd meg és uralkodj. Több dimenzióban.

3.6. Statikus keresőfa

Katasztrofális.

Van Emde Boas fák. Dinamikus eset szuperbonyolult, asszem.

3.7. Edénysor szita

Katasztrofális.

Az edénysor szität a szakdolgozatomban írtam le. Ez egy eratoszthenészi rekurzívan szegmentált szita.

Az eratoszthenészi sziták prímek többszöröseit jelölik meg egy adott intervallumon, és egy-egy prím többszöröseinek megjelöléseit a többi prímtől függetlenül végzik. A szität lehet használni prímek keresésére is, és az intervallum számainak prím-faktorizációjára is.

Egy szegmentált szita a teljes intervallumot kisebb intervallumokra osztja, ezek a szegmensek, és a szegmensekben külön-külön végzi el a megjelöléseket. A szegmensek átfedés nélkül lefedik a teljes intervallumot. A szegmens méretet úgy választják meg, hogy szitáláshoz használt prímek és egy szegmens a gyorsítótárban elférjen. Egy szegmentált szita cache-aware.

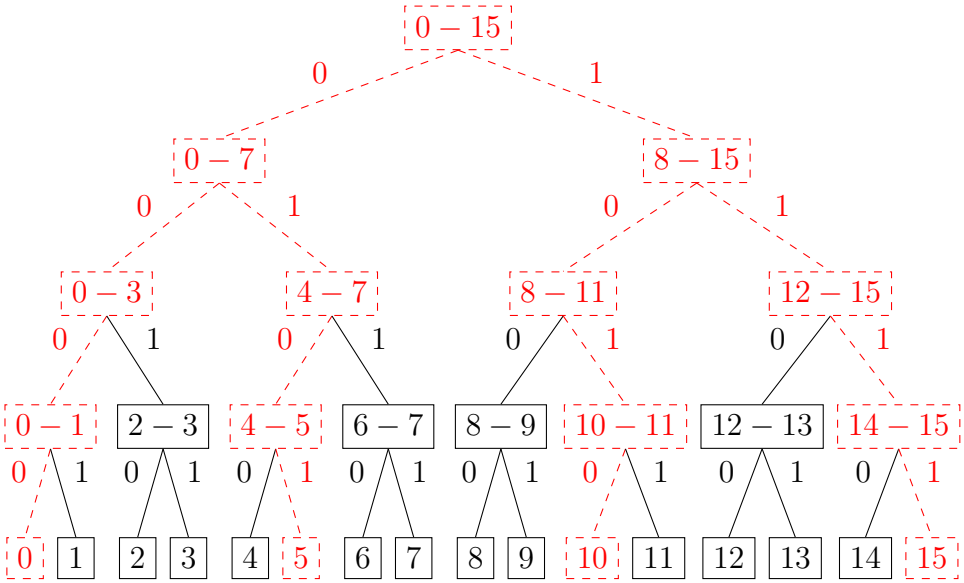
A rekurzív edényszita egy a^h hosszú intervallumot szitál, ahol a és h természetes számok, $a \geq 2$. Ha $h > 0$, akkor az intervallumot a darab ugyanolyan hosszú részre osztja, és ezeket rekurzívan szitálja. A prímek listáját előszűri, hogy egy szegmensbe csak olyan prím kerüljön, ami ténylegesen szitál is a szegmensben.

3.7.1. A rekurzív fa

Vegyük a $[u, u+a^h)$ intervallumot, u természetes szám. és egy p prímet, aminek van nullánál nagyobb többszöröse az intervallumban. Tegyük fel, hogy $h > 0$. Ekkor az i. részintervallum $[u + ia^{h-1}, u + (i+1)a^{h-1}), (i \in 0, ..., a-1)$

- Ha $p \geq a^h$, akkor p pontosan egy részintervallumban szitál.
- Ha $a^{h-1} < p < a^h$, akkor p legalább egy részintervallumban szitál, de nem feltétlen mindegyikben.
- Ha $p \leq a^{h-1}$, akkor p minden részintervallumban szitál.

Példának nézzük meg, hogy az 5 milyen utat jár be a rekurzív fában a $[0, 2^4)$ intervallum szitálásakor. Az ábrán a prím 0-t is bejárja, ami prímek keresésekor nem következik be.



3.1. ábra. Az 5 többszörösei a $[0, 2^4)$ intervallumhoz tartozó rekurzív fában.

3.7.2. Feladat

Jelöljük meg az $[u, u+a^h)$ intervallumban az összes olyan számot, amik többszörösei valamelyik $p \leq m$ prímnek.

3.7.3. A megoldás

Azt, hogy egy adott p prím melyik részintervallumban fog szitálni legközelebb, meghatározhatjuk maradékos osztással. Legyen u a szóba jöhető legkisebb intervallum kezdőpontja. Legyen $m = u \bmod p$. Ekkor p következő többszöröse

$$\begin{cases} u & \text{ha m} = 0 \\ u - m + p & \text{különben} \end{cases} \tag{3.1}$$

Az is megoldás, ha minden prím mellé tároljuk a legutolsó megtalált többszörösét is, és ezt frissítjük, amikor a prímet megjelölésre használjuk. Ezzel az osztás időigényét memóriaigényre cseréljük.

Algoritmus 3.1 Az $[u, u + a^h)$ intervallum szitálása

Require: a: a számrendszer alapja
Require: h: a fa magassága
Require: m: a keresett prímek felső korlátja
Require: u: a az intervallum kezdőpontja
Require: P: lista, azok a p prímszámok, hogy létezik p-nek többszöröse az $[u, u + a^h)$ intervallumban
Ensure: P: P kiegészítve az intervallumban talált prímekkel

```
function RekurzívSzita(a, h, m, u, P) → P
    if h = 0 then
        if P üres és u ≥ 2 then                    ▷ u egy újonnan megtalált prím
            if u ≤ m then
                P ← P ∪ {u}
            end if
        else
            jelöljük meg u-t                       ▷ u összetett szám
        end if
    else
        Legyenek a részintervallumok I_i = [u + ia^{h−1}, u + (i + 1)a^{h−1}, i ∈ [0, a)
        Q legyen listák egy a elemű tömbje, és legyen minden lista üres
        for p ∈ P do
            Legyen i a legkisebb olyan szám, hogy p-nek van többszöröse I_i-ben
            Q[i] ← Q[i] ∪ {p}
            P ← P \ {p}
        end for
        for i = 0, ..., a − 1 do
            Q[i] ← RekurzívSzita(a, h − 1, m, u + ia^{h−1}, Q[i])
            for p ∈ Q[i] do
                if p-nek van többszöröse I_{i+1}, ..., I_{a−1}-ben then
                    Legyen j a legkisebb olyan szám,
                    hogy p-nek van többszöröse I_{i+1}, ..., I_{a−1}-ben
                    Q[j] ← Q[j] ∪ {p}
                else
                    P ← P ∪ {p}
                end if
                Q[i] ← Q[i] \ {p}
            end for
        end for
    end if
    return P
end function
```

Prímek keresését a $[0, a^h)$ intervallumban a RekurzívSzita$(a, h, \lfloor\sqrt{(a^h)}\rfloor, a^h, \emptyset)$ hívás után az intervallum végigolvasásával lehet elvégezni. Azok a számok a prímek, amik nincsenek megjelölve.

A $[0, a^h)$ intervallum prím-faktorizációját a RekurzívSzita$(a, h, a^h, a^h, \emptyset)$ hívással lehet elvégezni. A hívás után az intervallum egy száma prím, ha nincs megjelölve, és összetett, ha meg van jelölve, és az összes prímosztója megjelölte.

3.7.4. Műveletigény gyorsítótár nélkül

Tegyük fel, hogy a $[0, a^h)$ intervallumot szitáljuk. Legyen $n = a^h$, és m a minket érdeklő prímek felső korlátja. Legyen $p \leq m$ egy prím.

Nézzük a rekurzív fában az összes a^k hosszú részintervallumot, $k \leq h$.

- Ha $p > a^k$, akkor p-nek legfeljebb egy többszöröse van egy adott részintervallumban, és összesen $\lfloor a^h \rfloor$ többszöröse van, és ennyi a^k hosszú részintervallumot jár be.
- Ha $p \leq a^k$, akkor p minden részintervallumban szitál.

$$p \leq a^k \Leftrightarrow log_a p \leq k$$

Így a rekurzív fában p legfeljebb a felső $h - \lfloor log_a p \rfloor + 1$ szinten jár be minden

csúcsot. Az alsó $\lfloor log_a p \rfloor$ szinten szintenként $\lfloor a^h/p \rfloor$ csúcsot jár be. Tehát p legfeljebb

$$
\begin{aligned}
C_p &\leq \sum_{k=0}^{h-\lfloor \log_a p \rfloor} a^k + \sum_{k=0}^{\lfloor log_a p \rfloor} \left\lfloor \frac{a^h}{p} \right\rfloor \\
&\leq \sum_{k=0}^{h-\lfloor \log_a p \rfloor} a^k + \sum_{k=0}^{\lfloor log_a p \rfloor} \frac{a^h}{p} \\
&= \sum_{k=0}^{h-\lfloor \log_a p \rfloor} a^k + \frac{(\lfloor log_a p \rfloor + 1)a^h}{p} \\
&\leq \sum_{k=0}^{h-\lfloor \log_a p \rfloor} a^k + \frac{(log_a p + 1)a^h}{p} \\
&= \sum_{k=0}^{h-\lfloor \log_a p \rfloor} a^k + n\left(\frac{log_a p}{p} + \frac{1}{p}\right) \\
&= \sum_{k=0}^{h-\lfloor \log_a p \rfloor} a^k + \frac{n}{\ln a}\frac{lnp}{p} + n\frac{1}{p} \\
&= \frac{a^{h-\lfloor \log_a p \rfloor+1} - 1}{a-1} + \frac{n}{\ln a}\frac{lnp}{p} + n\frac{1}{p} \\
&< \frac{a^{h-\lfloor \log_a p \rfloor+1}}{a-1} + \frac{n}{\ln a}\frac{lnp}{p} + n\frac{1}{p} \\
&\leq \frac{2a^{h-\lfloor \log_a p \rfloor+1}}{a} + \frac{n}{\ln a}\frac{lnp}{p} + n\frac{1}{p} \\
&= 2a^{h-\lfloor \log_a p \rfloor} + \frac{n}{\ln a}\frac{lnp}{p} + n\frac{1}{p} \\
&= \frac{2a^h}{a^{\lfloor \log_a p \rfloor}} + \frac{n}{\ln a}\frac{lnp}{p} + n\frac{1}{p} \\
&\leq \frac{2a^h}{a^{\log_a p - 1}} + \frac{n}{\ln a}\frac{lnp}{p} + n\frac{1}{p} \\
&= \frac{2a^{h+1}}{a^{\log_a p}} + \frac{n}{\ln a}\frac{lnp}{p} + n\frac{1}{p} \\
&= \frac{2a^{h+1}}{p} + \frac{n}{\ln a}\frac{lnp}{p} + n\frac{1}{p} \\
&= 2an\frac{1}{p} + \frac{n}{\ln a}\frac{lnp}{p} + n\frac{1}{p} \\
&= (2an+1)\frac{1}{p} + \frac{n}{\ln a}\frac{lnp}{p}
\end{aligned}
$$

csúcsot jár be a rekurzív fában.

Az összes prím által meglátogatott csúcsok számának becsléséhez felhasználjuk Mertens két tételét. Hivatkozás: wikipedia.

$$\sum_{p \le n} \frac{\ln p}{p} = \ln n + O(1)$$
$$\sum_{p \le n} \frac{1}{p} = \ln \ln n + O(1)$$

Így az összes prím által meglátogatott összes csúcs száma

$$\begin{aligned}
C &\le \sum_{p \le m} C_p \\
&= \sum_{p \le m} \left((2an+1)\frac{1}{p} + \frac{n}{\ln a}\frac{lnp}{p} \right) \\
&= \sum_{p \le m} (2an+1)\frac{1}{p} + \sum_{p \le m} \frac{n}{\ln a}\frac{lnp}{p} \\
&= (2an+1) \sum_{p \le m} \frac{1}{p} + \frac{n}{\ln a} \sum_{p \le m} \frac{lnp}{p} \\
&= (2an+1)\left(\ln \ln n + O(1)\right) + \frac{n}{\ln a}\left(\ln n + O(1)\right) \\
&= (2an+1)\ln \ln n + (2an+1)O(1) + \frac{n}{\ln a}\ln n + \frac{n}{\ln a}O(1) \\
&= \frac{n \ln n}{\ln a} + (2an+1)\ln \ln n + (2an+1)O(1) + \frac{n}{\ln a}O(1) \\
&= \frac{n \ln n}{\ln a} + (2an+1)\ln \ln n + O(an) + O(\frac{n}{\ln a}) \\
&= \frac{n \ln n}{\ln a} + (2an+1)\ln \ln n + O(an)
\end{aligned}$$

A RekurzívSzita program egy adott belső csúcsában egy adott prímmel többször is foglalkozik.

- Kiosztja legalább egy részintervallumra. Intervallumonként $O(1)$ időben.
- Összegyűjti a visszatérési értékbe. Egyszer $O(1)$ időben.

Valamint vannak a-tól függő inicializálási lépések, $O(a)$ időben. Egy levél csúcsban minden prímmel $O(1)$ időt foglalkozik.

Az összesítéshez számoljuk el a a részintervallumra kiosztás költségét a részintervallum csúcsában. Igy

- Egy levélben minden prímmel $O(1)$ időt foglalkozik.
- Egy belső csúcsban $O(a)$ időt tölt karbantartással, és prímenként $O(1)$ időt azzal, hogy rá osztották, vagy visszatér vele.

Tehát a teljes időigény

$$
\begin{aligned}
T &= \frac{a^h - 1}{a - 1} O(a) + CO(1) \\
&= \frac{a^h - 1}{a - 1} O(a) + O\left(\frac{n \ln n}{\ln a}\right) \\
&= O(a^h) + O\left(\frac{n \ln n}{\ln a}\right) \\
&= O(n) + O\left(\frac{n \ln n}{\ln a}\right) \\
&= O\left(\frac{n \ln n}{\ln a}\right)
\end{aligned}
$$

3.7.5. Cache-oblivious edényszita

Cache oblivious talán csak $a = 2$-re. A c program nem szép most tetszőleges a-ra. Lesz midly cache aware is, mert könnyű.

Megadunk egy konkrét implementációt a fenti RekurzívSzitára, ahol a listák egybefüggő memóriatartományok.

Az algoritmus a prímeket, amik egynél több, de nem minden részintervallumban szitálnak, minden rekurzív hívásnál újrarendezi.

3.7.6. Műveletigénye gyorsítótárral

Itt szégyenérzet nélkül fel fogom tenni, hogy egy intervallumban szitáló prímek száma az az egy-egy prím szitálásának várható értékének összege. Ami nem igaz, de senki se tud jobbat, és ad valami "elméletit", amivel össze lehet hasonlítani méréseket.

3.7.7. Feltételek a gyorsítótár méretére

3.7.8. Tetszőleges n-ig, nem csak teljes fára

3.7.9. Az igazi program fix méretű számokkal

3.7.10. Az igazi program változó méretű számokkal

Valszeg nem lesz, mert semmi hasznosat nem tudok mondani.

4. fejezet

Szimulátor

4.1. Igazi cache

Asszociativitás. Több szint. Több mag osztozik. Léteznek egészen speciális cache-ek, TLB, decoded microcode, trace cache, ...

4.2. Megoldás

4.3. Választható cache-ek

- utasítás/adat/mindkettő
- MIN/OPT.
- LRU
- FIFO, LIFO, LFU, ..., kevésbé izgalmasak
- teljes/1-2-3...-n asszociativitás
- mindenféle méretben
- egymás után akár több is

5. fejezet

Mérési eredmények

6. fejezet

Irodalomjegyzék

[1] Frigo, M., Leiserson, C. E., Prokop, H., Ramachandran, S.: Cache-oblivious algorithms, ACM Transactions on Algorithms (TALG), 8(1) (2012)., 1-22.

[2] Belady, L. A.: A study of replacement algorithms for virtual storage computers. IBM Systems Journal 5, 2 (1966), 78–101.

[3] Belady, L. A.; Nelson, R. A.; Shedler, G. S.: An anomaly in space-time characteristics of certain programs running in a paging machine. Communications of the ACM, 12(6), (1969) 349–353.

[4] Fornai, P., Iványi, A.: FIFO anomaly is unbounded. (2010) arXiv preprint arXiv:1003.1336.

[5] Mattson, R.L., Gecsei, J., Slutz, D.R., Traiger, I.L.: Evaluation Techniques for Storage Hierarchies. IBM Syst. J., 9, (1970) 78-117.

[6] Daniel D. Sleator and Robert E. Tarjan.: Amortized efficiency of list update and paging rules. Commun. ACM 28, 2 (Feb. 1985), 202–208.